

Network Security and Management

Term Project

AES 加解密演算法實作

系級：資工四甲

學號：B1129008

姓名：謝柏毅

Contents

1	操作說明	2
1.1	編譯方式	2
1.2	使用 Makefile 執行	2
1.3	Trace 分解模式	3
2	程式架構與功能	4
2.1	main.c : 流程控制	4
2.2	aes.h : AESContext	5
2.3	aes.c : AES 核心	6
2.4	Key Expansion	7
2.5	單一 block 加解密	7
3	ECB 平行化與 Trace 設計	8
4	檔案處理與例外處理	9
5	程式特色	10
6	心得	10
7	參考資料	10

1 操作說明

本程式以 C 語言實作 AES 加解密演算法，運作模式為 ECB (Electronic Codebook)。程式不使用現成加密函式庫，而是自行實作 AES 的 key expansion、SubBytes、ShiftRows、MixColumns、AddRoundKey、加密流程、解密流程與 trace 分解模式。專案透過 `Makefile` 編譯與執行，使用者可指定加密或解密、輸入檔、金鑰檔、輸出檔、執行緒數量與 trace 輸出。

1.1 編譯方式

在專案目錄執行：

```
1 make
```

`Makefile` 的主要設定如下：

```
1 CC = gcc
2 TARGET = aes
3 SRC = main.c aes.c file.c
4 WARN_FLAGS = -std=c11 -Wall -Wextra -pedantic
5 OPT_FLAGS = -O3
6 ACCEL_FLAGS = -DUSE_SIMD -mssse3 -fopenmp
7 CFLAGS = $(WARN_FLAGS) $(OPT_FLAGS) $(ACCEL_FLAGS)
```

其中 `-O3` 用於最佳化，`-mssse3` 啟用 SSSE3 SIMD 指令，`-fopenmp` 啟用 OpenMP 平行化。`SRC` 將主程式、AES 核心與檔案 I/O 模組一起編譯，因此使用者只要執行一次 `make`，就能產生完整的 `./aes` 執行檔。若程式碼有更新，再次執行 `make` 時，`Make` 也會依照檔案時間重新編譯需要更新的目標。

1.2 使用 `Makefile` 執行

`Makefile` 以變數控制執行參數。使用時可以先打開 `Makefile`，依照要執行的功能修改下列預設值，存檔後再執行 `make run`。

```
1 THREADS ?= 4
2 MODE ?= -e
3 INPUT ?= 測試資料/ciphertext2.dat
4 KEY ?= 測試資料/key128b.bin
5 OUTPUT ?= out_e.dat
6 TRACE ?= trace_parallel.txt
7 TRACE_ARGS = $(if $(TRACE),-x $(TRACE),)
```

各變數用途如下。`MODE=-e` 代表加密，`MODE=-d` 代表解密；`INPUT` 是輸入檔，可以是明文或密文；`KEY` 是金鑰檔；`OUTPUT` 是輸出檔；`THREADS` 是 OpenMP 執行緒數量。`TRACE` 是分解模式輸出檔名，只要 `TRACE` 有設定檔名，`Makefile` 就會透過 `TRACE_ARGS` 自動在執行指令中加入 `-x <trace_file>`，程式便會把每個 block 在每個 AES round 的 state 輸出到該檔案。若 `TRACE` 留空，`TRACE_ARGS` 會變成空字串，程式就只執行一般加解密，不輸出中間 round 結果。

以 `TRACE ?= encrypt_trace.txt` 為例，`make run` 實際上會近似展開成下列指令：

```
1 OMP_NUM_THREADS=4 ./aes -e -x encrypt_trace.txt 測試資料/plaintext2.dat 測試資料
  /key128b.bin cipher_output.dat
```

因此使用者操作時不需要每次手動輸入完整 `./aes` 指令，只要先修改 `Makefile` 的變數，存檔後執行 `make run` 即可。這也讓加密、解密、不同 key 長度與 trace 模式的切換更直觀。

例如要執行 AES-128 加密，可先把 `Makefile` 修改成：

```
1 THREADS ?= 4
2 MODE ?= -e
3 INPUT ?= 測試資料/plaintext2.dat
4 KEY ?= 測試資料/key128b.bin
5 OUTPUT ?= cipher_output.dat
6 TRACE ?=
```

接著執行：

```
1 make run
```

若要解密，則修改 `Makefile`：

```
1 MODE ?= -d
2 INPUT ?= cipher_output.dat
3 KEY ?= 測試資料/key128b.bin
4 OUTPUT ?= plain_output.dat
5 TRACE ?=
```

再執行：

```
1 make run
```

指定 AES-192 或 AES-256 時，只要更換 `KEY`，例如：

```
1 KEY ?= 測試資料/key192b.bin
```

或：

```
1 KEY ?= 測試資料/key256b.bin
```

1.3 Trace 分解模式

若要輸出每一輪 round 的中間 state，需要在 `Makefile` 中設定 `TRACE`。例如：

```
1 MODE ?= -e
2 INPUT ?= 測試資料/plaintext2.dat
3 KEY ?= 測試資料/key128b.bin
4 OUTPUT ?= cipher_output.dat
5 TRACE ?= encrypt_trace.txt
```

修改並儲存 `Makefile` 後執行：

```
1 make run
```

此時實際執行時會自動加入 `-x encrypt_trace.txt`，程式會將每個 block 的 initial state、AddRoundKey 後 state、各 round 結束後 state，以及 final round state 輸出到 `encrypt_trace.txt`。這個檔案可用來觀察 AES 每一輪如何逐步改變 state，適合用於驗證演算法流程是否符合規格，也符合本作業要求的「分解加解密模式」。

如果不想啟用 trace，可將 `TRACE` 留空：

```
1 TRACE ?=
```

Trace 格式如下：

```
1 block=0 round=00 input          00112233445566778899aabbccddeeff
2 block=0 round=00 add_round_key  0f1571c947d9e8590cb7add6af7f6798
3 block=0 round=01 round_end      ...
4 block=0 round=10 final_round_end ...
```

每行記錄 block 編號、round 編號、狀態名稱與 16 bytes state 的十六進位內容。清除執行檔可使用 `make clean`。實際使用時，若只是測量加解密效能，將 `TRACE` 留空，因為 trace 模式會額外產生大量文字輸出，會影響執行時間；若要檢查 AES 每一輪是否照標準流程執行，再開啟 `TRACE` 觀察輸出檔即可。

2 程式架構與功能

本專案主要由 `main.c`、`aes.c`、`aes.h`、`file.c`、`file.h` 與 `Makefile` 組成。`main.c` 負責參數解析、流程控制與效能輸出；`aes.c` 是 AES 核心；`aes.h` 定義 AES 常數與資料結構；`file.c` 負責二進位檔案讀寫；`Makefile` 管理編譯與執行。這樣的切分讓每個檔案責任明確：主程式不需要知道 AES round 的細節，AES 核心也不需要處理命令列參數或檔案開啟問題，降低模組之間的耦合。

```
1 命令列參數 -> main.c 解析模式與檔案路徑
2           -> file.c 讀取 input/key
3           -> main.c 檢查 input size 與 key size
4           -> aes.c 建立 AESContext 並產生 round keys
5           -> aes.c 執行 ECB 加密或解密
6           -> main.c 輸出 bytes/second
7           -> file.c 寫出結果檔案
```

2.1 main.c：流程控制

程式使用 `ProgramOptions` 保存本次執行模式與檔案路徑，避免後續流程直接操作 `argv`。

```
1 typedef enum {
2     MODE_ENCRYPT,
3     MODE_DECRYPT
4 } Mode;
5
6 typedef struct {
7     Mode mode;
8     const char *input_file;
9     const char *key_file;
```

```

10     const char *output_file;
11     const char *trace_file;
12     int trace_enabled;
13 } ProgramOptions;

```

命令列格式為：

```

1 ./aes -e [-x trace_file] input_file key_file output_file
2 ./aes -d [-x trace_file] input_file key_file output_file

```

解析後，程式會讀取輸入檔與 key 檔，並檢查 key 長度與輸入大小。AES block size 固定為 16 bytes，因此輸入檔必須非零且為 16 bytes 的倍數。Key 只接受 16、24、32 bytes，分別對應 AES-128、AES-192、AES-256。

```

1 if (key->size != AES_128_KEY_SIZE &&
2     key->size != AES_192_KEY_SIZE &&
3     key->size != AES_256_KEY_SIZE) {
4     return 0;
5 }
6
7 if (input->size == 0 || input->size % AES_BLOCK_SIZE != 0) {
8     return 0;
9 }

```

效能量測使用 `omp_get_wtime()`，只計算 AES 核心加解密時間，最後輸出處理資料量、秒數與 bytes/second。這裡刻意不把讀檔、寫檔時間放入 AES 核心效能，是為了讓輸出數據更接近演算法本身的處理速度，也方便比較不同 `THREADS` 數量或不同 key 長度造成的效能差異。

```

1 start_time = omp_get_wtime();
2 aes_encrypt_ecb_normal(input->data, output, input->size, aes_ctx);
3 end_time = omp_get_wtime();
4 bytes_per_second = (double)processed_bytes / elapsed_seconds;

```

2.2 aes.h : AESContext

`aes.h` 定義 AES 的基本常數與 context。AES-128、AES-192、AES-256 的 round 數分別為 10、12、14。

```

1 #define AES_BLOCK_SIZE 16
2 #define AES_128_KEY_SIZE 16
3 #define AES_192_KEY_SIZE 24
4 #define AES_256_KEY_SIZE 32
5 #define AES_128_ROUNDS 10
6 #define AES_192_ROUNDS 12
7 #define AES_256_ROUNDS 14

```

```

1 typedef struct {
2     size_t key_size;
3     int key_words;
4     int rounds;
5     uint8_t round_keys[AES_MAX_EXPANDED_KEY_SIZE];
6 } AESContext;

```

`AESContext` 會保存 key 大小、key words 數量、round 數與 key expansion 後的 round keys。這樣加密與解密時就能依照不同 AES 版本使用正確 round 數。AES-128 的 key words 為 4，因此共有 10 rounds；AES-192 的 key words 為 6，共 12 rounds；AES-256 的 key words 為 8，共 14 rounds。主程式只要呼叫 `aes_init()`，後續加解密函式就能從 `ctx->rounds` 得知要執行幾輪，並從 `ctx->round_keys` 取出每一輪所需的 key。

2.3 aes.c : AES 核心

AES 加密主要包含 `SubBytes`、`ShiftRows`、`MixColumns`、`AddRoundKey`。解密則使用反向操作 `InvSubBytes`、`InvShiftRows`、`InvMixColumns`。AES 的 state 是 4 x 4 bytes 矩陣，但在程式中以一維陣列儲存，排列方式為 column-major order，也就是同一個 column 的 4 bytes 在記憶體中連續存放。這點會影響 `ShiftRows` 與 `MixColumns` 的索引設計，因此程式中特別以固定 shuffle mask 和 column rotate 的方式處理。

`AddRoundKey` 是將 16 bytes state 與 16 bytes round key 做 XOR。此程式使用 SIMD 一次處理整個 block：

```
1 static void add_round_key(uint8_t *state, const uint8_t *round_key) {
2     __m128i state_vec = _mm_loadu_si128((const __m128i *)state);
3     __m128i key_vec = _mm_loadu_si128((const __m128i *)round_key);
4     __m128i result_vec = _mm_xor_si128(state_vec, key_vec);
5     _mm_storeu_si128((__m128i *)state, result_vec);
6 }
```

`SubBytes` 使用 S-box 替換每個 byte，提供 AES 的非線性：

```
1 static void sub_bytes(uint8_t *state) {
2     for (int i = 0; i < AES_BLOCK_SIZE; i++) {
3         state[i] = sbox[state[i]];
4     }
5 }
```

`ShiftRows` 使用 `_mm_shuffle_epi8()` 完成 row 位移。AES state 採 column-major order，因此 shuffle mask 必須對應 state 的記憶體排列。

```
1 static void shift_rows(uint8_t *state) {
2     const __m128i mask = _mm_setr_epi8(
3         0, 5, 10, 15, 4, 9, 14, 3,
4         8, 13, 2, 7, 12, 1, 6, 11
5     );
6     __m128i s = _mm_loadu_si128((const __m128i *)state);
7     s = _mm_shuffle_epi8(s, mask);
8     _mm_storeu_si128((__m128i *)state, s);
9 }
```

`MixColumns` 在 $GF(2^8)$ 上混合每個 column。程式以 `xtime_vec()` 實作乘以 2，若最高位元溢位則 XOR `0x1b` 做 reduction。`MixColumns` 的作用是提高資料擴散效果：單一 byte 的改變會在後續 rounds 中影響更多位置。若只做 `SubBytes` 與 `ShiftRows`，資料雖然會被替換與位移，但 column 內部的混合仍不足；加入 `MixColumns` 後，每個 column 的 4 bytes 會互相影響，能更符合 AES 的擴散設計。

```
1 static inline __m128i xtime_vec(__m128i x) {
2     const __m128i msb_mask = _mm_set1_epi8((char)0x80);
```

```

3     const __m128i reduction = _mm_set1_epi8(0x1b);
4     __m128i high_bits = _mm_and_si128(x, msb_mask);
5     __m128i shifted = _mm_add_epi8(x, x);
6     __m128i needs_reduction = _mm_cmpeq_epi8(high_bits, msb_mask);
7     return _mm_xor_si128(shifted, _mm_and_si128(needs_reduction, reduction));
8 }

```

2.4 Key Expansion

`aes_init()` 依照 key 長度決定 AES 版本。16 bytes key 對應 AES-128，24 bytes 對應 AES-192，32 bytes 對應 AES-256。

```

1 switch (key_size) {
2     case AES_128_KEY_SIZE: ctx->key_words = 4; ctx->rounds = 10; break;
3     case AES_192_KEY_SIZE: ctx->key_words = 6; ctx->rounds = 12; break;
4     case AES_256_KEY_SIZE: ctx->key_words = 8; ctx->rounds = 14; break;
5     default: return 0;
6 }

```

Key expansion 的核心是 `schedule_core()`，會先做 word rotate，再做 S-box 替換，最後與 Rcon 做 XOR。

```

1 static void schedule_core(uint8_t word[4], int rcon_index) {
2     rotate_word_left(word);
3     sub_word(word);
4     word[0] ^= rcon[rcon_index];
5 }

```

AES-256 在特定位置需要額外執行 `sub_word()`，程式也依照 AES 規則處理。因此同一份程式能支援三種 key 長度。產生出的 `round_keys` 會依照 round 順序連續存放，每個 round 實際使用 16 bytes，讓加密與解密函式可透過 `ctx->round_keys + round * AES_BLOCK_SIZE` 取得指定 round 的 key。

2.5 單一 block 加解密

加密流程為：先做 initial `AddRoundKey`，中間 rounds 執行 `SubBytes` -> `ShiftRows` -> `MixColumns` -> `AddRoundKey`，最後一輪省略 `MixColumns`。

```

1 memcpy(state, input, AES_BLOCK_SIZE);
2 add_round_key(state, ctx->round_keys);
3
4 for (round = 1; round < ctx->rounds; round++) {
5     sub_bytes(state);
6     shift_rows(state);
7     mix_columns(state);
8     add_round_key(state, ctx->round_keys + round * AES_BLOCK_SIZE);
9 }
10
11 sub_bytes(state);
12 shift_rows(state);
13 add_round_key(state, ctx->round_keys + ctx->rounds * AES_BLOCK_SIZE);

```

解密流程與加密相反，先使用最後一組 round key，再由最後 round 往前還原。加密最後一輪不做 MixColumns，因此解密最後還原到 round 0 時也不做 InvMixColumns，這是 AES 標準流程中很重要的對應關係：

```
1 add_round_key(state, ctx->round_keys + ctx->rounds * AES_BLOCK_SIZE);
2
3 for (round = ctx->rounds - 1; round > 0; round--) {
4     inv_shift_rows(state);
5     inv_sub_bytes(state);
6     add_round_key(state, ctx->round_keys + round * AES_BLOCK_SIZE);
7     inv_mix_columns(state);
8 }
9
10 inv_shift_rows(state);
11 inv_sub_bytes(state);
12 add_round_key(state, ctx->round_keys);
```

3 ECB 平行化與 Trace 設計

ECB 模式會把輸入切成多個 16 bytes block，每個 block 獨立加解密，不需要 IV，也沒有 block 間相依性。因此本程式可用 OpenMP 平行處理多個 block。

```
1 #pragma omp parallel for
2 for (long block = 0; block < block_count; block++) {
3     size_t offset = (size_t)block * AES_BLOCK_SIZE;
4     aes_encrypt_block(input + offset, output + offset, ctx);
5 }
```

每個 thread 處理不同 offset 的 block，因此不會寫到相同輸出位置。此設計能利用多核心 CPU 提升大量資料處理速度。由於 ECB 不需要前一個 block 的輸出作為下一個 block 的輸入，所以平行化邏輯比 CBC 等模式單純，也符合本作業限制的 CPU 平行化要求。

Trace 模式需輸出每個 round 的 state。若多個 thread 直接寫同一個檔案，輸出順序可能混亂，因此程式為每個 block 建立獨立 TraceBuffer，平行處理時先寫入各自 buffer，最後再依 block index 順序寫入檔案。

```
1 typedef struct {
2     char *data;
3     size_t size;
4     size_t capacity;
5 } TraceBuffer;
```

```
1 #pragma omp parallel for schedule(static)
2 for (long block = 0; block < block_count; block++) {
3     traces[block].capacity = trace_capacity;
4     traces[block].data = (char *)malloc(trace_capacity);
5     traces[block].size = 0;
6     aes_encrypt_block_with_trace(input + offset, output + offset,
7                                 ctx, &traces[block], (size_t)block);
8 }
9
```

```

10 for (long block = 0; block < block_count; block++) {
11     fwrite(traces[block].data, 1, traces[block].size, trace);
12 }

```

此設計兼顧平行效能與輸出可讀性。Trace 每行以 `block=<index> round=<round> label state_hex` 表示，方便檢查每輪 AES state 的變化。加密 trace 會從 round 0 的 initial AddRoundKey 逐步記錄到最後 round；解密 trace 則從最後一組 round key 開始往 round 0 還原，因此可以清楚看出加密與解密流程方向相反。

在 normal 模式下，程式只輸出最後的密文或明文，因此速度較快；在 trace 模式下，程式除了完成加解密，還會記錄 initial state、AddRoundKey 後 state、中間 round state 與 final round state。這些資訊可以用來對照 AES 流程，確認最後一輪是否省略 MixColumns、解密流程是否正確反向執行，以及不同 key 長度下 round 數是否正確。

4 檔案處理與例外處理

`file.c` 以 `FileBuffer` 儲存二進位檔案內容與大小。

```

1 typedef struct {
2     uint8_t *data;
3     size_t size;
4 } FileBuffer;

```

讀檔使用 `open()`、`fstat()`、`read()`。先取得檔案大小，再配置記憶體，最後把檔案完整讀入 buffer。

```

1 fd = open(filename, O_RDONLY);
2 fstat(fd, &file_stat);
3 buffer->size = (size_t)file_stat.st_size;
4 buffer->data = (uint8_t *)malloc(buffer->size);

```

寫檔使用 `open()` 搭配 `O_WRONLY` | `O_CREAT` | `O_TRUNC`，代表建立或覆蓋輸出檔，並用迴圈確保所有 bytes 都寫入完成。

```

1 fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
2 while (bytes_written < size) {
3     result = write(fd, data + bytes_written, size - bytes_written);
4     bytes_written += (size_t)result;
5 }

```

程式處理的錯誤包含：參數格式錯誤、輸入檔無法開啟、key 檔無法開啟、輸出檔無法寫入、trace 檔無法建立、key 長度不合法、輸入大小不是 16 bytes 倍數、記憶體配置失敗。這些檢查能避免不合法輸入進入 AES 核心造成錯誤結果。特別是本程式不做 padding，因此若輸入檔不是完整 block，程式會直接輸出錯誤訊息，而不是自行補齊資料。錯誤流程中也會釋放已配置的 buffer，例如讀取 key 失敗時釋放 input、trace 檔開啟失敗時釋放 output，避免無效輸入造成記憶體洩漏。

效能輸出以 bytes/second 表示。此數值是用處理資料量除以 AES 核心執行時間得到，可用來比較不同 `THREADS` 數量、不同 key 長度或 trace/normal 模式下的速度差異。一般而言，trace 模式因為需要記錄大量中間狀態，速度會比 normal 模式慢，因此正式效能比較應以 normal 模式為主。

5 程式特色

本程式完成作業要求的 AES ECB 加解密，並支援 AES-128、AES-192、AES-256 三種金鑰長度。程式以 `Makefile` 統一編譯與執行方式，使用者能透過變數快速切換模式與測試資料。效能方面，ECB block 彼此獨立，因此使用 OpenMP 平行化；單一 block 內的部分操作則使用 SSSE3 SIMD intrinsic 加速，例如 `AddRoundKey`、`ShiftRows` 與 $GF(2^8)$ 運算。分解模式方面，程式可輸出每個 block 各 round state，並透過 per-block trace buffer 維持平行處理下的輸出順序。

整體設計重點可分成三個方向。第一是正確性：依照 AES 標準處理 rounds、key expansion 與 inverse rounds。第二是效能：利用 ECB block 獨立性做 OpenMP 平行化，並用 SIMD 減少單一 block 內的運算成本。第三是可觀察性：透過 trace 檔案輸出每輪 state，讓使用者不只看到最後結果，也能檢查中間流程。

6 心得

這次作業讓我更深入理解 AES 是由許多精確的數學步驟組合而成。剛開始我只知道 AES 包含 SubBytes、ShiftRows、MixColumns 與 AddRoundKey，但真正實作後才發現每個細節都會影響結果。例如 state 的排列方式採 column-major order，最後一輪不能做 MixColumns，解密時 round key 的使用順序也必須反過來。只要其中一個步驟寫錯，最後產生的密文或明文就會完全不同。

在實作過程中，我也體會到效能最佳化必須建立在正確性之上。ECB 模式中每個 block 彼此獨立，因此很適合使用 OpenMP 平行化，讓不同 CPU thread 同時處理不同 block。不過加入平行化後，trace 模式就不能直接讓所有 thread 同時寫同一個檔案，否則輸出順序會變得混亂。因此程式使用每個 block 各自的 buffer，最後再依照 block index 寫入 trace 檔，這讓我理解到平行程式除了追求速度，也必須注意結果是否穩定且容易閱讀。

這次作業也讓我練習了 SIMD intrinsic。像 AddRoundKey 原本可以用迴圈逐 byte XOR，但使用 128-bit SIMD 後可以一次處理 16 bytes。ShiftRows 也能用 shuffle mask 直接重排 state。這些最佳化讓我更理解底層硬體指令如何影響程式效能。不過 SIMD 程式比一般 C 程式更不直觀，因此撰寫清楚的註解與模組化函式也很重要。整體而言，這次作業讓我從密碼學流程、C 語言實作、檔案處理、錯誤處理、效能量測到平行化都有完整練習，也更能理解加密程式在實際系統中必須同時兼顧正確性、速度與可維護性。

7 參考資料

1. 平行程式 Lec4：共享記憶體程式設計 - OpenMP
<https://hackmd.io/@Jaychao2099/PP-2025-4>
2. 一文读懂 SIMD 指令集 目前最全 SSE/AVX 介绍
https://blog.csdn.net/qq_32916805/article/details/117637192